

# DIVISÃO E CONQUISTA

João Pedro Oliveira Batisteli



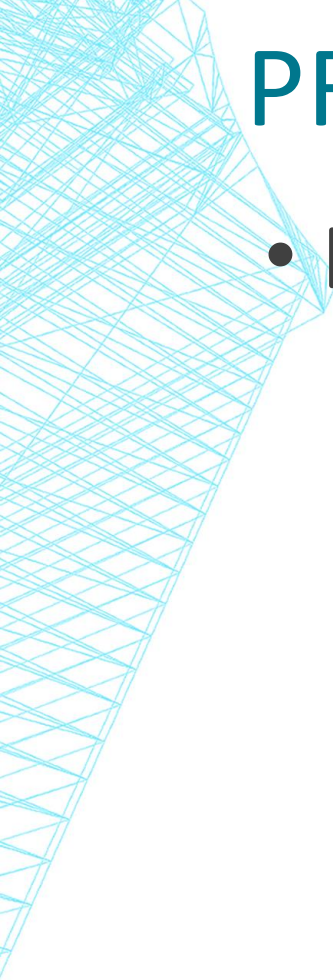
# ALGORITMOS E TEMPOS DE EXECUÇÃO

- Há classes de algoritmos pouco eficientes com um conjunto grande de entradas.

| n          | $f(n) = n^2$        |
|------------|---------------------|
| 100        | 10.000              |
| 1.000      | 1.000.000           |
| 10.000     | 100.000.000         |
| 100.000    | 10.000.000.000      |
| 1.000.000  | 1.000.000.000.000   |
| 10.000.000 | 100.000.000.000.000 |

# PROBLEMAS GRANDES, DIVIDIDOS ...

- Matematicamente, pode acontecer de a combinação das operações de soluções de partes menores do problema ser menor do que o número original de operações.



# PROBLEMAS GRANDES, DIVIDIDOS ...

- Ex: 1.000.000, dividido em 10 partes de 100.000.



# PROBLEMAS GRANDES, DIVIDIDOS ...

- Ex: 1.000.000, dividido em 10 partes de 100.000.

| n          | $f(n) = n^2$        |
|------------|---------------------|
| 100        | 10.000              |
| 1.000      | 1.000.000           |
| 10.000     | 100.000.000         |
| 100.000    | 10.000.000.000      |
| 1.000.000  | 1.000.000.000.000   |
| 10.000.000 | 100.000.000.000.000 |

# PROBLEMAS GRANDES, DIVIDIDOS ...

- Ex: 1.000.000, dividido em 10 partes de 100.000.

| n          | $f(n) = n^2$        |
|------------|---------------------|
| 100        | 10.000              |
| 1.000      | 1.000.000           |
| 10.000     | 100.000.000         |
| 100.000    | $10^{10}$           |
| 1.000.000  | $10^{12}$           |
| 10.000.000 | 100.000.000.000.000 |

# PROBLEMAS GRANDES, DIVIDIDOS ...

- Ex: 1.000.000, dividido em 10 partes de 100.000.

- $10 \times 10^{10} = 10^{11}$

- $10^{11} < 10^{12}$

| n          | f(n) = n <sup>2</sup> |
|------------|-----------------------|
| 100        | 10.000                |
| 1.000      | 1.000.000             |
| 10.000     | 100.000.000           |
| 100.000    | 10 <sup>10</sup>      |
| 1.000.000  | 10 <sup>12</sup>      |
| 10.000.000 | 100.000.000.000.000   |

# DIVISÃO E CONQUISTA

- Técnica de projeto de algoritmos que procura aumentar a eficiência da resolução de um problema **quebrando este problema em partes antes** de começar a solucioná-lo.



# DIVISÃO E CONQUISTA

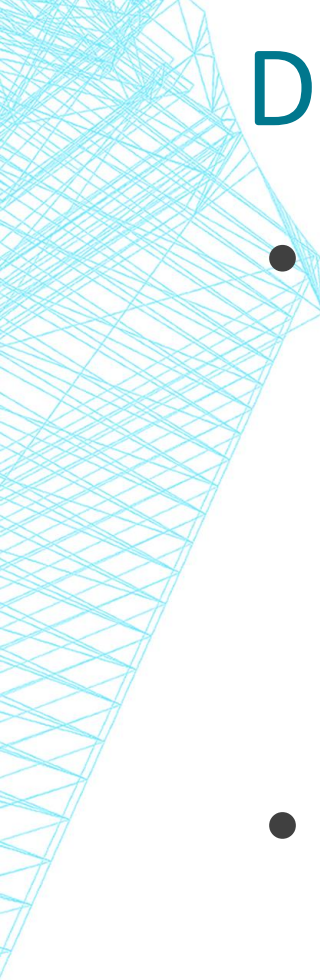
- Dividir o problema em partes menores,
- Encontrar soluções para essas partes,
- e combiná-las em uma solução global.

# DIVISÃO E CONQUISTA

- Algoritmos amplamente conhecidos:
  - Mergesort.
  - Quicksort.

# DIVISÃO E CONQUISTA: VANTAGENS

- Resolução de problemas **difíceis**;
- Geralmente leva a soluções **eficientes e elegantes**, principalmente se forem **recursivas**;
- Pode gerar algoritmos **eficientes**, com forte tendência a complexidade **logarítmica**;
- Requerem um número menor de acessos à memória.
- ***São altamente paralelizáveis.***



# DIVISÃO E CONQUISTA: PROBLEMAS

- **Recursão** ou pilha explícita:
  - Número de **chamadas recursivas** e/ou armazenadas na **pilha** pode ser um inconveniente.
- Dificuldade na seleção dos **casos-base**.
- **Repetição de subproblemas**.

# DIVISÃO E CONQUISTA: QUANDO USAR?

1. Deve ser possível decompor uma instância do problema em *sub-instâncias*.
2. As *sub-instâncias* devem ser balanceadas.
3. Não deve haver *sub-instâncias* sobrepostas.
4. A combinação dos resultados deve ser **eficiente**.



# ALGORITMO GERAL

Função Divisao\_E\_Conquista(x):

// 1. CASO BASE

se (x é pequeno o suficiente)

retornar Resolve\_Diretamente(x)

senao

// 2. DIVIDIR

decompor o problema x em n subproblemas:  $x_1, x_2, \dots, x_n$

// 3. CONQUISTAR (Resolver recursivamente)

para cada i de 1 até n:

$y_i = \text{Divisao\_E\_Conquista}(x_i)$

// 4. COMBINAR

combinar os resultados  $y_1, y_2, \dots, y_n$  em uma solução Y

retornar Y

# DIVISÃO, CONQUISTA E COMPLEXIDADE

- Algoritmos recursivos → equação de recorrência

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- Sendo:
  - a – número de subproblemas gerados
  - b – tamanho de cada um dos subproblemas
  - f(n) – custo para fazer a divisão e a combinação

## EXEMPLO: MAX/MIN

- Seja  $A$  um vetor de aleatório de inteiros,  $A[0..n-1]$ .
- Determine o maior e o menor elementos deste vetor;

## EXEMPLO: MAX/MIN

- Seja  $A$  um vetor de aleatório de inteiros,  $A[0..n-1]$ .
- Determine o maior e o menor elementos deste vetor;
- Já conhecemos:
  - Melhor caso:  $f(n) = n - 1$
  - Pior caso:  $f(n) = 2n - 2$

## EXEMPLO: MAX/MIN

- Seja  $A$  um vetor de aleatório de inteiros,  $A[0..n-1]$ .
- Determine o maior e o menor elementos deste vetor *usando divisão e conquista*;
- Determine a complexidade (número de comparações) para achar estes dois elementos, considerando que  $A$  possui  $n$  elementos.



# EXEMPLO: MAX/MIN

08

23

16

42

15

93

04

77

# EXEMPLO: MAX/MIN

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 08 | 23 | 16 | 42 | 15 | 93 | 04 | 77 |
|----|----|----|----|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 08 | 23 | 16 | 42 |
|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 15 | 93 | 04 | 77 |
|----|----|----|----|

# EXEMPLO: MAX/MIN

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 08 | 23 | 16 | 42 | 15 | 93 | 04 | 77 |
|----|----|----|----|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 08 | 23 | 16 | 42 |
|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 15 | 93 | 04 | 77 |
|----|----|----|----|

|    |    |  |    |    |
|----|----|--|----|----|
| 08 | 23 |  | 16 | 42 |
|----|----|--|----|----|

|    |    |  |    |    |
|----|----|--|----|----|
| 15 | 93 |  | 04 | 77 |
|----|----|--|----|----|

# EXEMPLO: MAX/MIN

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 08 | 23 | 16 | 42 | 15 | 93 | 04 | 77 |
|----|----|----|----|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 08 | 23 | 16 | 42 |
|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 15 | 93 | 04 | 77 |
|----|----|----|----|

|    |    |
|----|----|
| 08 | 23 |
|----|----|

Min 08

Max 23

|    |    |
|----|----|
| 16 | 42 |
|----|----|

Min 16

Max 42

|    |    |
|----|----|
| 15 | 93 |
|----|----|

Min 15

Max 93

|    |    |
|----|----|
| 04 | 77 |
|----|----|

Min 04

Max 77

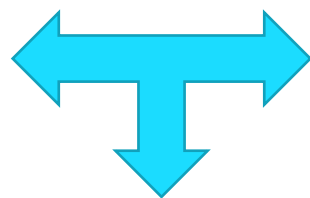
# EXEMPLO: MAX/MIN

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 08 | 23 | 16 | 42 | 15 | 93 | 04 | 77 |
|----|----|----|----|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 08 | 23 | 16 | 42 |
|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 08 | 23 | 16 | 42 |
|----|----|----|----|

Min 08      Min 16  
Max 23      Max 42

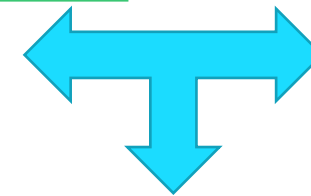


Min 08  
Max 42

|    |    |    |    |
|----|----|----|----|
| 15 | 93 | 04 | 77 |
|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 15 | 93 | 04 | 77 |
|----|----|----|----|

Min 15      Min 04  
Max 93      Max 77



Min 04  
Max 93

# EXEMPLO: MAX/MIN

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 08 | 23 | 16 | 42 | 15 | 93 | 04 | 77 |
|----|----|----|----|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 08 | 23 | 16 | 42 |
|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 15 | 93 | 04 | 77 |
|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 08 | 23 | 16 | 42 |
|----|----|----|----|

|    |    |    |    |
|----|----|----|----|
| 15 | 93 | 04 | 77 |
|----|----|----|----|

Min 08  
Max 23

Min 16  
Max 42

Min 15  
Max 93

Min 04  
Max 77

Min 08  
Max 42

Min 04  
Max 93

Min 04  
Max 93



# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8          MAXMIN4(Linf, Meio, Max1, Min1)
9          MAXMIN4(Meio+1, Lsup, Max2, Min2)
10         if Max1 > Max2
11             then Max ← Max1
12             else Max ← Max2
13         if Min1 < Min2
14             then Min ← Min1
15             else Min ← Min2
```

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8      MAXMIN4(Linf, Meio, Max1, Min1)
9      MAXMIN4(Meio+1, Lsup, Max2, Min2)
10     if Max1 > Max2
11         then Max ← Max1
12         else Max ← Max2
13     if Min1 < Min2
14         then Min ← Min1
15         else Min ← Min2
```

Se “x” é pequeno,  
resolvemos “x”

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8          MAXMIN4(Linf, Meio, Max1, Min1)
9          MAXMIN4(Meio+1, Lsup, Max2, Min2)
10         if Max1 > Max2
11             then Max ← Max1
12             else Max ← Max2
13         if Min1 < Min2
14             then Min ← Min1
15             else Min ← Min2
```

Em vez de descer a recursão até sobrar apenas 1 elemento (como no Mergesort), o caso base atua quando o subvetor tem **1 ou 2 elementos**

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8          MAXMIN4(Linf, Meio, Max1, Min1)
9          MAXMIN4(Meio+1, Lsup, Max2, Min2)
10         if Max1 > Max2
11             then Max ← Max1
12             else Max ← Max2
13         if Min1 < Min2
14             then Min ← Min1
15             else Min ← Min2
```

Se tiver apenas 1 elemento, ele é ao mesmo tempo o máximo e o mínimo.

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3        then Max ← A[Lsup]
4             Min ← A[Linf]
5        else Max ← A[Linf]
6             Min ← A[Lsup]
7    else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8        MAXMIN4(Linf, Meio, Max1, Min1)
9        MAXMIN4(Meio+1, Lsup, Max2, Min2)
10       if Max1 > Max2
11           then Max ← Max1
12           else Max ← Max2
13       if Min1 < Min2
14           then Min ← Min1
15           else Min ← Min2
```

Se tiver 2 elementos, o algoritmo faz uma única comparação ( $A[Linf] < A[Lsup]$ ) e já define quem é o maior e quem é o menor

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2  then if A[Linf] < A[Lsup]
3      then Max ← A[Lsup]
4          Min ← A[Linf]
5      else Max ← A[Linf]
6          Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8      MAXMIN4(Linf, Meio, Max1, Min1)
9      MAXMIN4(Meio+1, Lsup, Max2, Min2)
10     if Max1 > Max2
11         then Max ← Max1
12         else Max ← Max2
13     if Min1 < Min2
14         then Min ← Min1
15         else Min ← Min2
```

Isso é uma boa  
estratégia?



# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8      MAXMIN4(Linf, Meio, Max1, Min1)
9      MAXMIN4(Meio+1, Lsup, Max2, Min2)
10     if Max1 > Max2
11         then Max ← Max1
12     else Max ← Max2
13     if Min1 < Min2
14         then Min ← Min1
15     else Min ← Min2
```

Decomposição em 2 subconjuntos

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8      MAXMIN4(Linf, Meio, Max1, Min1)
9      MAXMIN4(Meio+1, Lsup, Max2, Min2)
10     if Max1 > Max2
11         then Max ← Max1
12         else Max ← Max2
13     if Min1 < Min2
14         then Min ← Min1
15         else Min ← Min2
```

Se o vetor tem mais de 2 elementos, ele simplesmente encontra o índice do meio (*Meio*) para quebrar o problema pela metade, exatamente como é feito na Busca Binária ou no Mergesort.

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup - Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8      MAXMIN4(Linf, Meio, Max1, Min1)
9      MAXMIN4(Meio+1, Lsup, Max2, Min2)
10     if Max1 > Max2
11         then Max ← Max1
12         else Max ← Max2
13     if Min1 < Min2
14         then Min ← Min1
15         else Min ← Min2
```

Se o vetor tem mais de 2 elementos, ele simplesmente encontra o índice do meio (*Meio*) para quebrar o problema pela metade, exatamente como é feito na Busca Binária ou no Mergesort.

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8          MAXMIN4(Linf, Meio, Max1, Min1)
9          MAXMIN4(Meio+1, Lsup, Max2, Min2)
10         if Max1 > Max2
11             then Max ← Max1
12             else Max ← Max2
13         if Min1 < Min2
14             then Min ← Min1
15             else Min ← Min2
```

Combinação das soluções

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8          MAXMIN4(Linf, Meio, Max1, Min1)
9          MAXMIN4(Meio+1, Lsup, Max2, Min2)
10         if Max1 > Max2
11             then Max ← Max1
12             else Max ← Max2
13         if Min1 < Min2
14             then Min ← Min1
15             else Min ← Min2
```

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8          MAXMIN4(Linf, Meio, Max1, Min1)
9          MAXMIN4(Meio+1, Lsup, Max2, Min2)
10     if Max1 > Max2
11         then Max ← Max1
12         else Max ← Max2
13     if Min1 < Min2
14         then Min ← Min1
15         else Min ← Min2
```

Para descobrir o **Máximo Global** daquele pedaço, ele compara o campeão da esquerda (*Max1*) com o campeão da direita (*Max2*). O vencedor é guardado na variável *Max*.

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8          MAXMIN4(Linf, Meio, Max1, Min1)
9          MAXMIN4(Meio+1, Lsup, Max2, Min2)
10         if Max1 > Max2
11             then Max ← Max1
12             else Max ← Max2
13         if Min1 < Min2
14             then Min ← Min1
15             else Min ← Min2
```

Para descobrir o **Mínimo Global**, ele compara o perdedor da esquerda (*Min1*) com o perdedor da direita (*Min2*). O menor deles é guardado na variável *Min*.



# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup – Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8          MAXMIN4(Linf, Meio, Max1, Min1)
9          MAXMIN4(Meio+1, Lsup, Max2, Min2)
10         if Max1 > Max2
11             then Max ← Max1
12             else Max ← Max2
13         if Min1 < Min2
14             then Min ← Min1
15             else Min ← Min2
```

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup - Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3         then Max ← A[Lsup]
4              Min ← A[Linf]
5         else Max ← A[Linf]
6              Min ← A[Lsup]
7    else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8         MAXMIN4(Linf, Meio, Max1, Min1)
9         MAXMIN4(Meio+1, Lsup, Max2, Min2)
10        if Max1 > Max2
11            then Max ← Max1
12            else Max ← Max2
13        if Min1 < Min2
14            then Min ← Min1
15            else Min ← Min2
```

$T(1) = 1$  comparação

# MAX/MIN COM DIVISÃO E CONQUISTA

MAXMIN4(*Linf*, *Lsup*, *Max*, *Min*)

▷ Variáveis auxiliares: *Max1*, *Max2*, *Min1*, *Min2*, *Meio*

```
1  if (Lsup - Linf) ≤ 1
2    then if A[Linf] < A[Lsup]
3          then Max ← A[Lsup]
4              Min ← A[Linf]
5          else Max ← A[Linf]
6              Min ← A[Lsup]
7  else Meio ←  $\lfloor \frac{Linf + Lsup}{2} \rfloor$ 
8        MAXMIN4(Linf, Meio, Max1, Min1)
9        MAXMIN4(Meio+1, Lsup, Max2, Min2)
10       if Max1 > Max2
11         then Max ← Max1
12         else Max ← Max2
13       if Min1 < Min2
14         then Min ← Min1
15         else Min ← Min2
```

$T(1) = 1$  comparação

$$T(n) = T(n/2) + T(n/2) + 2$$

$$T(n) = 2T(n/2) + 2$$

# MAX/MIN COM DIVISÃO E CONQUISTA

- $T(1) = 1$
- $T(n) = 2T(n/2) + 2$
- A considerar:
  - Variáveis locais na recursividade.
  - Comparação da condição de parada

# MAX/MIN COM DIVISÃO E CONQUISTA

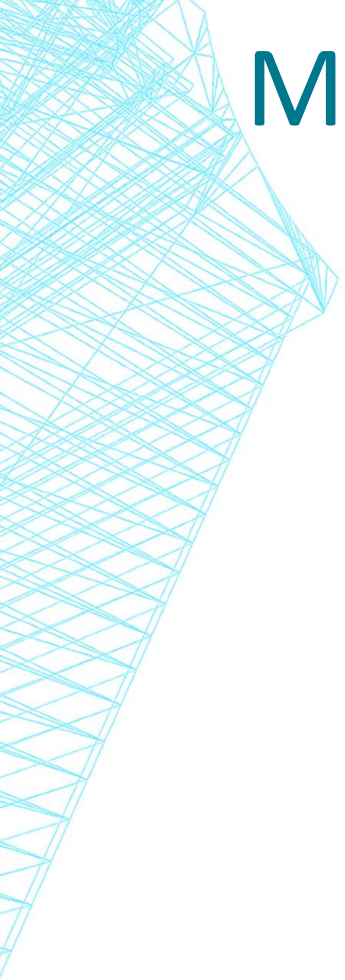
se (x é pequeno o suficiente)  
retornar `Resolve_Diretamente(x)`

- O que é “pequeno o suficiente” ?
- Ex:
  - Mergesort/quicksort com 14 milhões de elementos
  - $\log_2(n) = 24$
  - Com 9 divisões, chegamos a 27.344 elementos.

# MAX/MIN COM DIVISÃO E CONQUISTA

se (x é pequeno o suficiente)  
retornar `Resolve_Diretamente(x)`

- O que é “pequeno o suficiente” ?
- Ex:
  - Mergesort/quicksort com 14 milhões de elementos
  - $\log_2(n) = 24$
  - Com  $\approx 24$  divisões, chegamos a 1 elemento.



# MAX/MIN COM DIVISÃO E CONQUISTA

***“A combinação dos resultados deve ser eficiente.”***



# DIVISÃO E CONQUISTA: DECISÕES

*combinar os resultados  $y_1, y_2, \dots, y_n$  em uma solução  $Y$*

- Como combinar as soluções ?
- Ex:
  - Mergesort: intercalação de vizinhos.

# EXEMPLO: ORDENAÇÃO MERGESORT

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 4 | 8 | 3 | 6 | 5 | 2 | 7 |
|---|---|---|---|---|---|---|---|

|   |   |   |   |
|---|---|---|---|
| 1 | 4 | 8 | 3 |
|---|---|---|---|

|   |   |   |   |
|---|---|---|---|
| 6 | 5 | 2 | 7 |
|---|---|---|---|

|   |   |
|---|---|
| 1 | 4 |
|---|---|

|   |   |
|---|---|
| 8 | 3 |
|---|---|

|   |   |
|---|---|
| 6 | 5 |
|---|---|

|   |   |
|---|---|
| 2 | 7 |
|---|---|

|   |
|---|
| 1 |
|---|

|   |
|---|
| 4 |
|---|

|   |
|---|
| 8 |
|---|

|   |
|---|
| 3 |
|---|

|   |
|---|
| 6 |
|---|

|   |
|---|
| 5 |
|---|

|   |
|---|
| 2 |
|---|

|   |
|---|
| 7 |
|---|

|   |   |
|---|---|
| 1 | 4 |
|---|---|

|   |   |
|---|---|
| 3 | 8 |
|---|---|

|   |   |
|---|---|
| 5 | 6 |
|---|---|

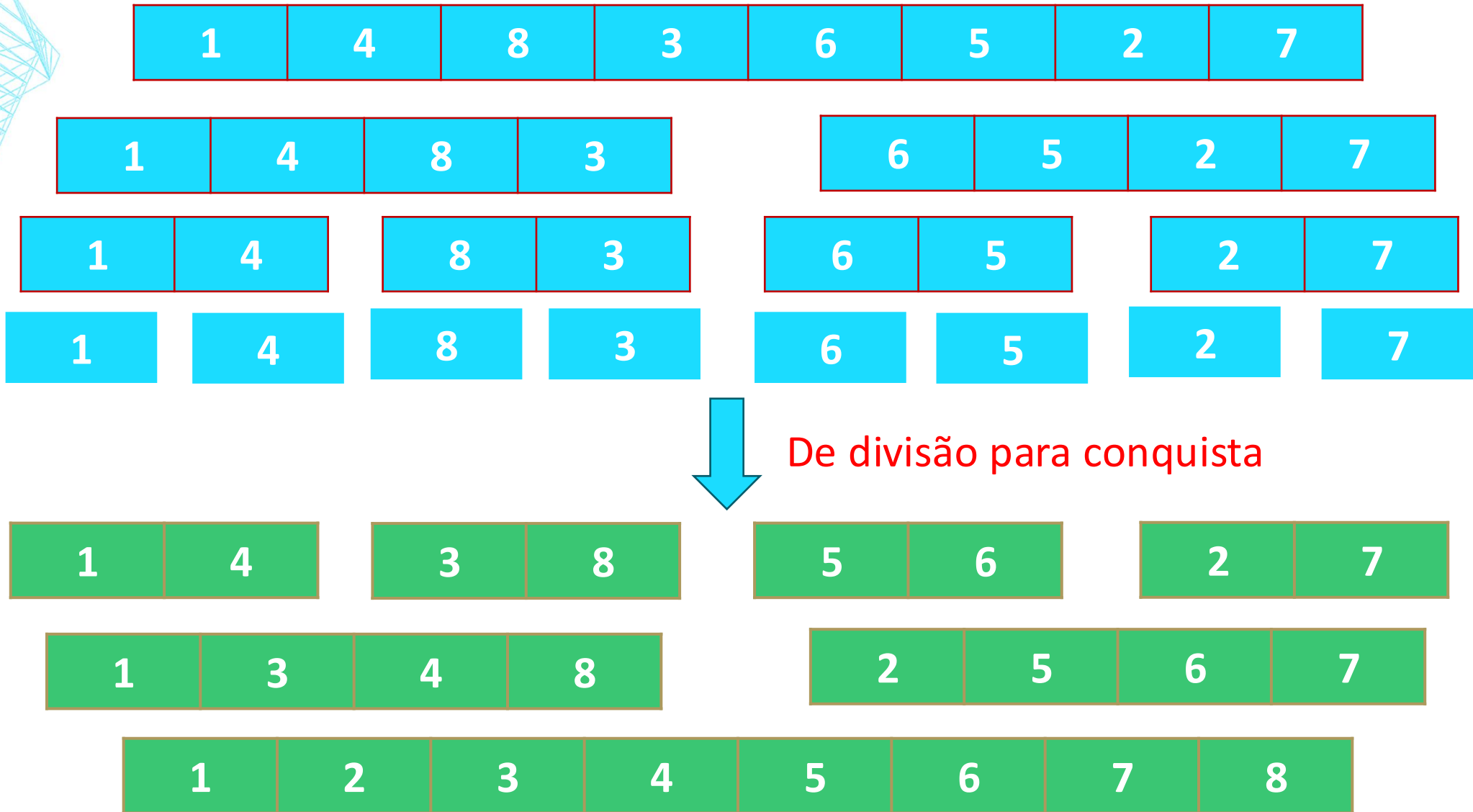
|   |   |
|---|---|
| 2 | 7 |
|---|---|

|   |   |   |   |
|---|---|---|---|
| 1 | 3 | 4 | 8 |
|---|---|---|---|

|   |   |   |   |
|---|---|---|---|
| 2 | 5 | 6 | 7 |
|---|---|---|---|

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|

# EXEMPLO: ORDENAÇÃO MERGESORT



# DIVISÃO E CONQUISTA: DECISÕES

*combinar os resultados  $y_1, y_2, \dots, y_n$  em uma solução  $Y$*

- Como combinar as soluções ?
- Ex:
  - Quicksort: partição e troca do pivô já resolvem

# EXEMPLO: QUICKSORT

|    |    |   |   |    |    |    |    |
|----|----|---|---|----|----|----|----|
| 23 | 93 | 8 | 4 | 15 | 77 | 16 | 42 |
|----|----|---|---|----|----|----|----|



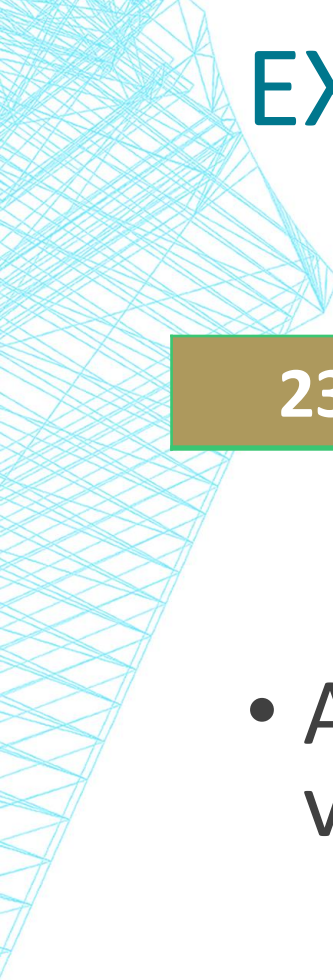
Part



i

- Item  $i < \text{pivô}$ ?
  - Se sim, avançar Part, trocar  $i$  com  $Part$  e avançar  $i$
  - Se não, avançar  $i$
- Ao final, avançar Part e trocar  $\text{pivô}$  com  $Part$

# EXEMPLO: QUICKSORT



|    |   |   |    |    |    |    |    |
|----|---|---|----|----|----|----|----|
| 23 | 8 | 4 | 15 | 16 | 42 | 93 | 77 |
|----|---|---|----|----|----|----|----|

- Agora, executamos recursivamente nos dois trechos do vetor, à esquerda e à direita do pivô.
- A conquista é “imediata” no retorno.

# EXEMPLO: SUBSEQUÊNCIA DE SOMA MÁXIMA

Dado um conjunto de dados (*array*), qual subsequência contínua de seus itens gera um valor máximo?

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|



# EXEMPLO: SUBSEQUÊNCIA DE SOMA MÁXIMA

Dado um conjunto de dados (*array*), qual subsequência contínua de seus itens gera um valor máximo?

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

- Divisão do Vetor

# EXEMPLO: SUBSEQUÊNCIA DE SOMA MÁXIMA

Dado um conjunto de dados (*array*), qual subsequência contínua de seus itens gera um valor máximo?

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

- Divisão do Vetor

# EXEMPLO: SUBSEQUÊNCIA DE SOMA MÁXIMA

Dado um conjunto de dados (*array*), qual subsequência contínua de seus itens gera um valor máximo?

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

- Divisão do Vetor
- Como avaliar e combinar os resultados?

# EXEMPLO: SUBSEQUÊNCIA DE SOMA MÁXIMA

Dado um conjunto de dados (*array*), qual subsequência contínua de seus itens gera um valor máximo?

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

- Divisão do Vetor
- Maior de cada metade?

# EXEMPLO: SUBSEQUÊNCIA DE SOMA MÁXIMA

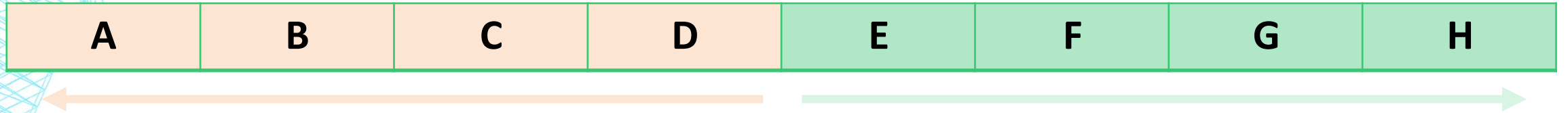
Dado um conjunto de dados (*array*), qual subsequência contínua de seus itens gera um valor máximo?

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

- Como combinar soluções?

13

# SUBSEQUÊNCIA DE SOMA MÁXIMA



# SUBSEQUÊNCIA DE SOMA MÁXIMA

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|





# SUBSEQUÊNCIA DE SOMA MÁXIMA

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|



|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 |   |    |    |   |
|    |   |     |   | 3 | -1 | -2 | 8 |

# SUBSEQUÊNCIA DE SOMA MÁXIMA

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|



|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 |   |    |    |   |
|    |   |     |   | 3 | -1 | -2 | 8 |

O resultado pode estar na interseção

# SUBSEQUÊNCIA DE SOMA MÁXIMA

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|



|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 |   |    |    |   |
|    |   |     |   | 3 | -1 | -2 | 8 |

**Ou na união do resultado de diferentes segmentos**

# SUBSEQUÊNCIA DE SOMA MÁXIMA

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 | 3 | -1 | -2 | 8 |
|----|---|-----|---|---|----|----|---|

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|

|    |   |     |   |   |    |    |   |
|----|---|-----|---|---|----|----|---|
| 10 | 2 | -20 | 5 |   |    |    |   |
|    |   |     |   | 3 | -1 | -2 | 8 |

Combinação:

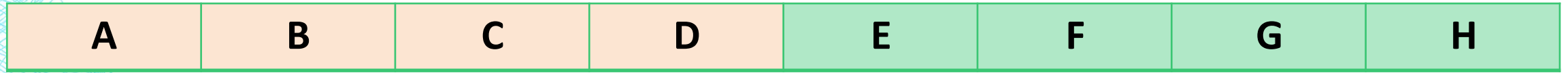
MAIOR A-D

MAIOR E-H

MAIOR INTERSECÇÃO

**Ou na união do resultado de diferentes segmentos**

# SUBSEQUÊNCIA DE SOMA MÁXIMA



# MAX/MIN : COMBINAÇÃO (INTERSEÇÃO)

Função `Maior_Soma_Cruzando_Meio(A, inicio, meio, fim)`:

`//1. Expandindo para a esquerda`

`soma_esquerda = -INFINITO`

`soma_atual = 0`

`para i de meio descendo até inicio:`

`soma_atual = soma_atual + A[i]`

`se soma_atual > soma_esquerda:`

`soma_esquerda = soma_atual`

`// 2. Expandindo para a direita`

`soma_direita = -INFINITO`

`soma_atual = 0`

`para j de (meio + 1) até fim:`

`soma_atual = soma_atual + A[j]`

`se soma_atual > soma_direita:`

`soma_direita = soma_atual`

`// 3. A maior intersecção é a união dos dois melhores lados`

`retornar (soma_esquerda + soma_direita)`

# MAX/MIN : DIVISÃO E CONQUISTA

```
Função Subsequencia_Soma_Maxima(A, inicio, fim):  
    // CASO BASE: O vetor tem apenas 1 elemento  
    se inicio == fim:  
        retornar A[inicio]  
    // DIVIDIR: Encontrar o ponto médio  
    meio = piso((inicio + fim) / 2)  
    // CONQUISTAR: Resolver recursivamente para as duas metades  
    maior_esq = Subsequencia_Soma_Maxima(A, inicio, meio)  
    maior_dir = Subsequencia_Soma_Maxima(A, meio + 1, fim)  
    // COMBINAR: Calcular a maior soma que cruza o meio  
    maior_cruzamento = Maior_Soma_Cruzando_Meio(A, inicio, meio, fim)  
    // DECISÃO FINAL: Retornar o maior valor entre as 3 possibilidades  
    se (maior_esq >= maior_dir) E (maior_esq >= maior_cruzamento):  
        retornar maior_esq  
    senao se (maior_dir >= maior_esq) E (maior_dir >= maior_cruzamento):  
        retornar maior_dir  
    senao:  
        retornar maior_cruzamento
```

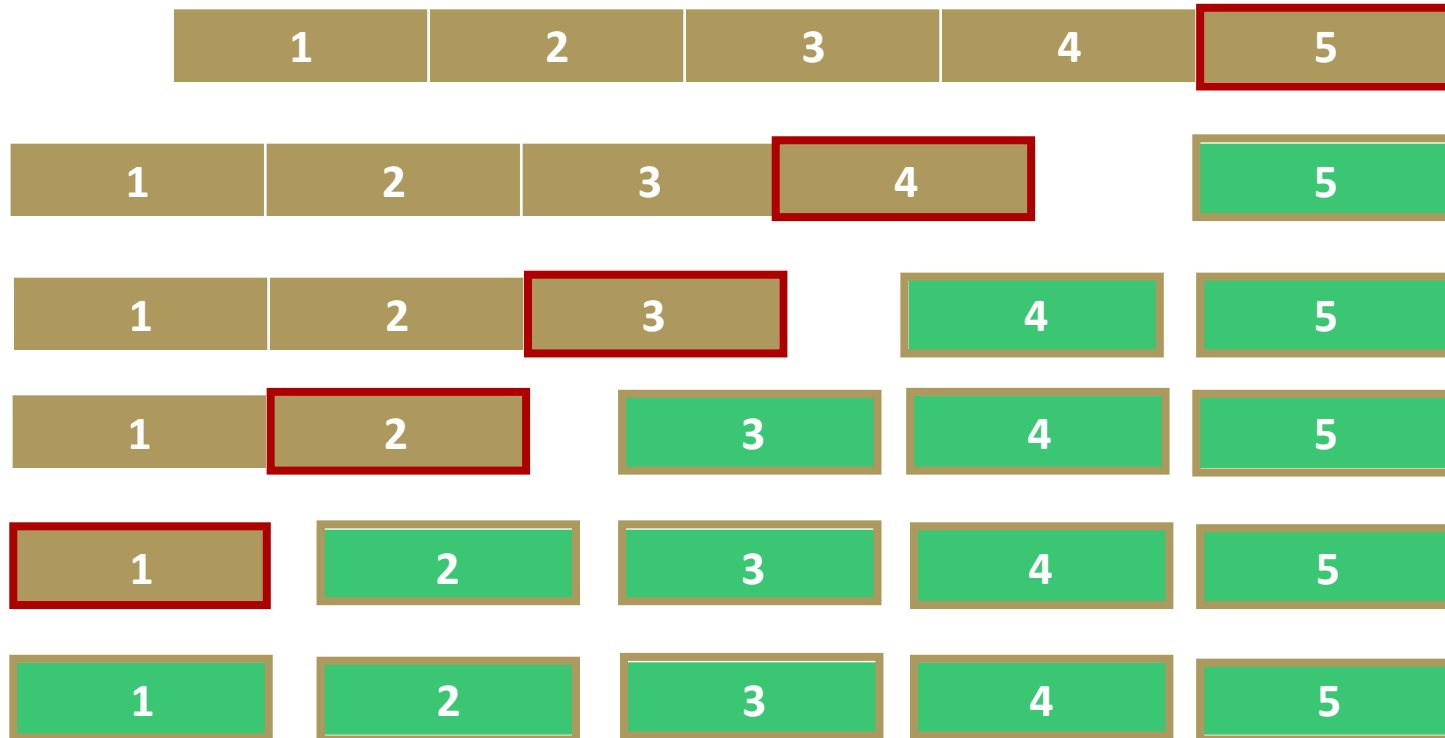


# DIVISÃO E CONQUISTA

- Outros algoritmos conhecidos:
  - Mediana;
  - Multiplicação de números grandes (algoritmo de Karatsuba);
  - Menor distância entre um par de pontos;
  - Multiplicação de matrizes (algoritmo de Strassen);
  - Contador de inversões em conjuntos numéricos

# DIVISÃO E CONQUISTA: BALANCEAMENTO

- É importante procurar sempre manter o balanceamento na subdivisão de um problema em partes menores.
- Ex: Pior caso do quicksort.

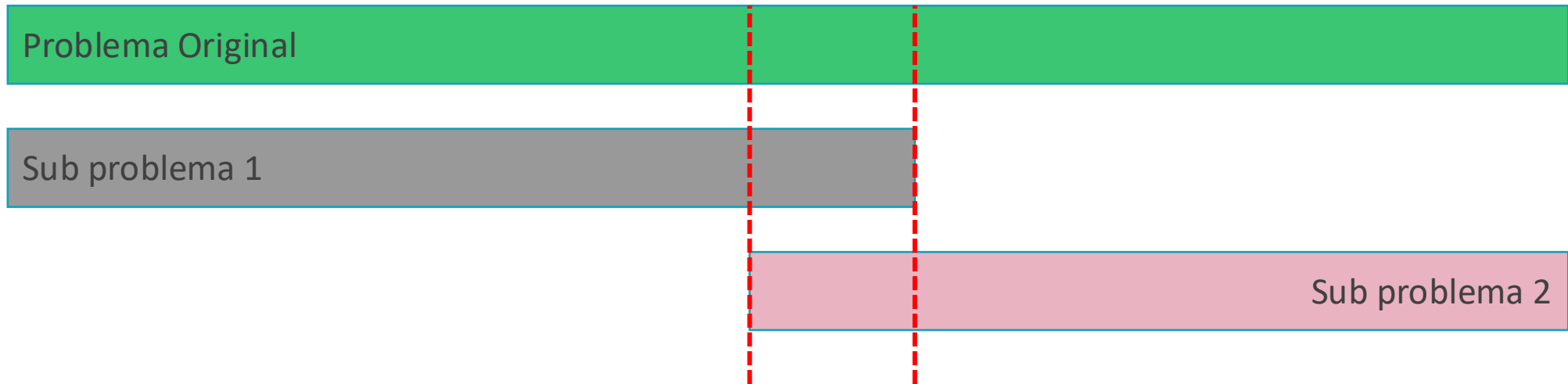


# DIVISÃO E CONQUISTA: BALANCEAMENTO

- Subproblemas com sobreposição de instâncias levam algoritmos de divisão e conquista à ineficiência.

# DIVISÃO E CONQUISTA: SOBREPOSIÇÃO

- Subproblemas com sobreposição de instâncias levam algoritmos de divisão e conquista à ineficiência.



# DIVISÃO E CONQUISTA: BALANCEAMENTO

- Subproblemas com sobreposição de instâncias levam algoritmos de divisão e conquista à ineficiência.
- Ex: Fibonacci recursivo.

# EXEMPLO: FIBONACCI

- Crie um método recursivo para retornar o *n-ésimo* elemento da sequência de Fibonacci.
  - $F = 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89 \dots$

$$\text{Fib}(n) = \begin{cases} 1 & , \text{ se } n \leq 2 \\ \text{Fib}(n - 1) + \text{Fib}(n - 2) & , \text{ se } n > 2 \end{cases}$$



# EXEMPLO: FIBONACCI

```
int Fib(int n){  
    if (n <= 2) return 1;  
    return Fib(n - 1) + Fib(n - 2);  
}
```

$$\text{Fib}(n) = \begin{cases} 1 & , \text{se } n \leq 2 \\ \text{Fib}(n - 1) + \text{Fib}(n - 2) & , \text{se } n > 2 \end{cases}$$

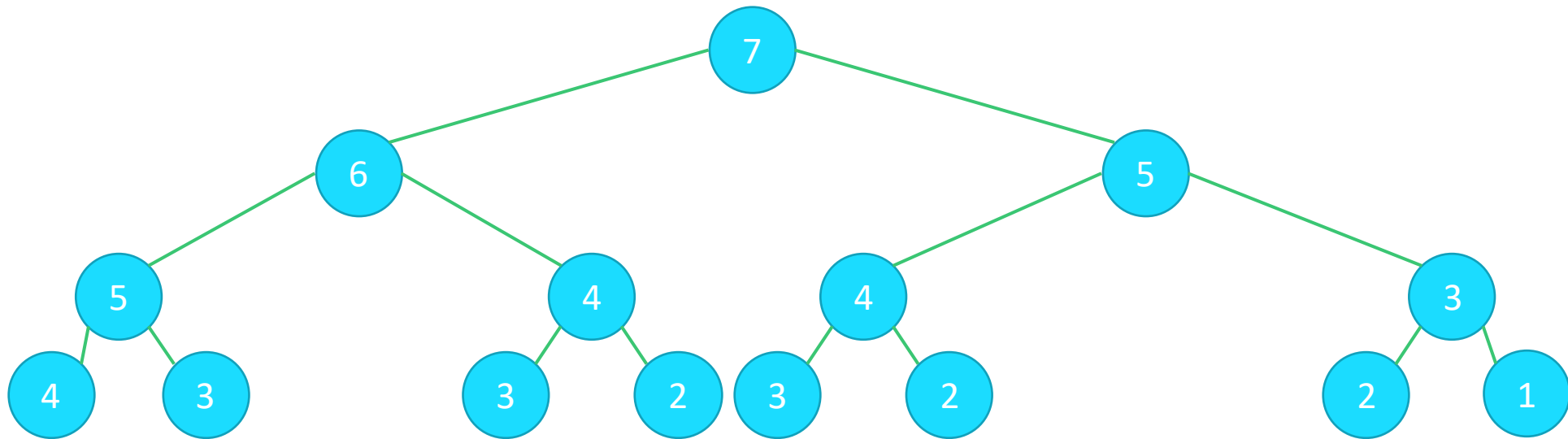
# ESEMPLO: FIBONACCI

```
int Fib(int n){  
    if (n <= 2) return 1;  
    return Fib(n - 1) + Fib(n - 2);  
}
```

$$\begin{aligned}\text{fib}(7) &= \text{fib}(6) + \text{fib}(5) \\ &= \text{fib}(5) + \text{fib}(4) \\ &= \text{fib}(4) + \text{fib}(3)\end{aligned}$$

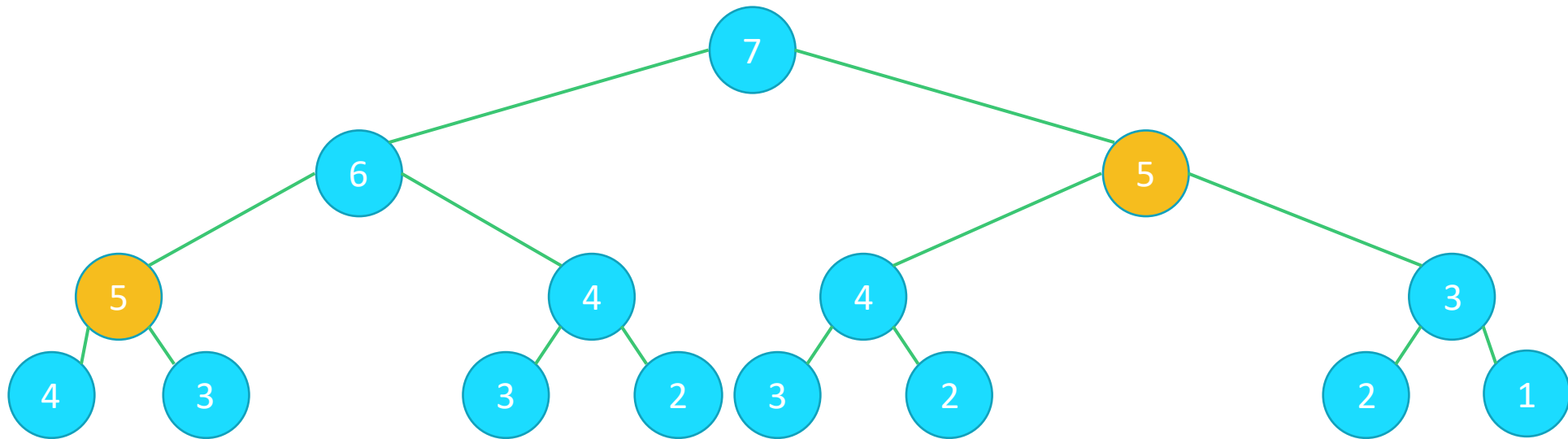
# EXEMPLO: FIBONACCI

```
int Fib(int n){  
    if (n <= 2) return 1;  
    return Fib(n - 1) + Fib(n - 2);  
}
```



# ESEMPLO: FIBONACCI

```
int Fib(int n){  
    if (n <= 2) return 1;  
    return Fib(n - 1) + Fib(n - 2);  
}
```

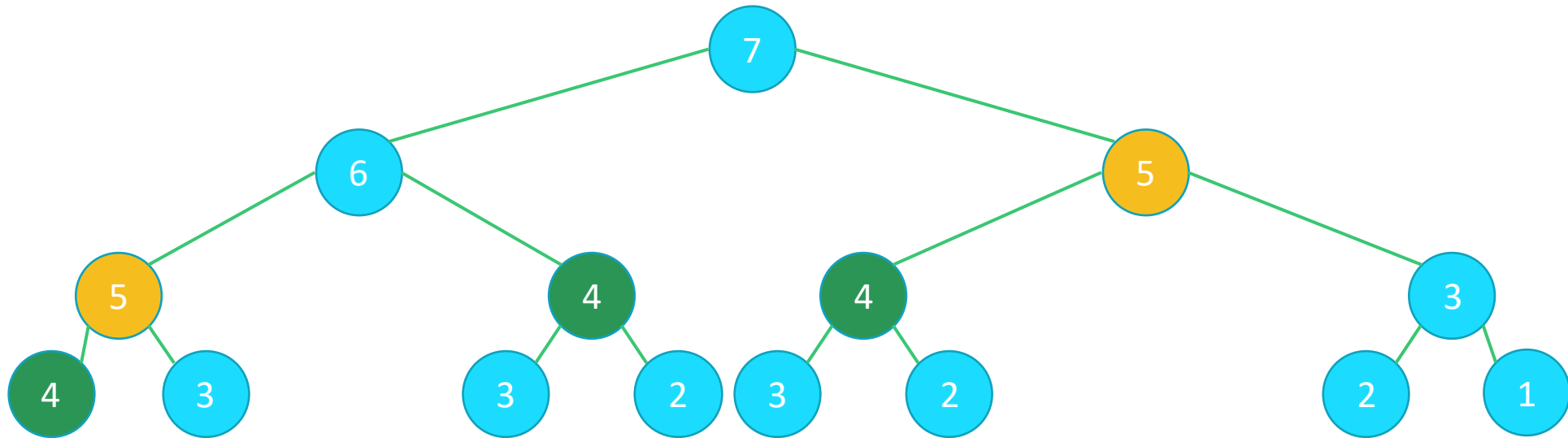


EX

in

}

```
int Fib(int n){
    if (n <= 2) return 1;
    return Fib(n - 1) + Fib(n - 2);
}
```



EX

in

}

```
int Fib(int n){
    if (n <= 2) return 1;
    return Fib(n - 1) + Fib(n - 2);
}
```

